Sistema del jugador

Guía de implementación y uso | Revisión: 6 de noviembre de 2026

Estado: implementado; la verificación de movimiento y aspecto se realiza en Unity.

Fecha de revisión: 6 de noviembre de 2026.

Esta guía explica el jugador local de NIWI-MU: cómo se mueve, cómo mantiene su estado, cómo se representa por capas y cómo recibe el nombre, rol y progreso de la sesión.

¿Qué es?

El jugador es el GameObject que recibe controles locales y participa en las interacciones del mundo. El prefab de referencia es `Assets/_SocialProyectoCenigraf/Prefabs/Entities/PlayerDemo/Player.prefab`. Está formado por componentes de física, movimiento, estado, aspecto y orden de dibujo.

El jugador usa dos estados Redux relacionados, pero distintos:

- **`PlayerStateData` / `PlayerStateStore`:** estado operativo del avatar: posición, movimiento, orientación, skin, colores, tamaño y offset del collider, orden Y e identidad del jugador. La autoridad local o remota se define en `PlayerReplicationContext`.
- **`GameSessionStateData` / `GameSessionStore`:** datos de la sesión actual: nombre, rol seleccionado, nivel y experiencia. El store sobrevive a cambios de escena durante la ejecución.

El flujo de una actualización del avatar es:

1. Un controlador o sistema envía una acción al store.
2. El reducer calcula un nuevo estado inmutable con los valores normalizados.
3. `PlayerStateStore` publica `StateChanged` cuando el valor cambia.
4. `PlayerStateRuntimeApplier` actualiza el cuerpo, collider y orden Y; `PlayerLayeredSkinController` actualiza sprites, animación, dirección y color.

Así el movimiento, la apariencia y los componentes visuales observan un solo estado operativo. El nivel, nombre y rol de la sesión se mantienen en el store de sesión y se muestran en el HUD. El inventario y las misiones son sistemas separados.

¿Para qué se usa?

Se usa para:

- Leer controles de teclado, mando o joystick táctil y mover el avatar con colisiones.
- Mantener la posición, orientación, rol, skin, animación, apariencia y collider en un estado común.
- Renderizar un personaje animado a partir de capas de sprites y ajustar el orden según el movimiento hacia el frente o el fondo.
- Mantener los datos de sesión entre escenas y actualizar el HUD con nombre, rol, nivel y experiencia.
- Distinguir el jugador local de una réplica remota dentro de la interfaz de replicación.

`PlayerReplicationBridge` ofrece un límite neutral para capturar y aplicar snapshots, pero no conecta a un servidor ni implementa transporte de red. La sincronización de red real requiere otra integración.

¿Cómo se usa?

1. Colocar y configurar el jugador local

1. Usa `Assets/_SocialProyectoCenigraf/Prefabs/Entities/PlayerDemo/Player.prefab` como base del jugador local.
2. Asegura que el objeto raíz tenga el tag `Player`.
3. Conserva en el mismo objeto `PlayerStateStore`, `PlayerStateRuntimeApplier`, `PlayerMovementController`, `Rigidbody2D`, `BoxCollider2D`, `WorldYSort` y `PlayerReplicationContext`.
4. Conserva el hijo `Visual/LayeredSkin` y los renderers de las capas. `PlayerLayeredSkinController` necesita un `PlayerSkinCatalog`, un `PlayerLayerOrderProfile` y referencias a los renderers.
5. Comprueba que el jugador esté activo en cada escena donde se espera que el jugador local pueda moverse. El flujo actual de teleport utiliza al jugador que ya está en la escena destino; no lo instancia ni transfiere el anterior.

`PlayerMovementController` requiere `Rigidbody2D`, `BoxCollider2D` y `PlayerStateStore`. `PlayerStateRuntimeApplier` requiere además `WorldYSort`. La configuración física de referencia usa un `Rigidbody2D` cinemático, gravedad cero, collider no trigger y orden Y activo.

2. Controlar el movimiento

El controlador busca la acción de Input System `Player/Move`:

- **Teclado:** WASD o flechas.
- **Mando:** stick izquierdo o D-pad, según los bindings de `Assets/InputSystem_Actions.inputactions`.
- **Pantalla táctil:** el joystick que crea `MobileControls` en plataformas móviles.

En `PlayerMovementController`:

Campo	Uso
`Move Speed`	**Velocidad solicitada del jugador. El prefab `Player` actual usa 2.**
`Collision Mask`	**Capas incluidas en las consultas de colisión.**
`Skin Width`	**Separación mínima que se conserva al resolver una colisión.**

El movimiento se ejecuta en `FixedUpdate`. El controlador usa consultas `BoxCast` en los ejes X e Y por separado para detenerse en paredes y permitir deslizarse a lo largo de ellas. Las zonas trigger no bloquean el desplazamiento. Si hay un `GameplayInputLock` activo o el jugador es una réplica remota, la entrada se limpia y el personaje no acepta control local.

No muevas el jugador cambiando el `Transform` mientras el controlador está activo. Para teletransportar o reconciliar la posición, actualiza el store con la acción correspondiente y sincroniza el `Rigidbody2D`; los sistemas de teleport ya lo hacen.

3. Entender y modificar el estado del jugador

El estado se lee y modifica con `PlayerStateStore`. Usa sus métodos para cambios habituales; también se pueden enviar acciones explícitas al reducer.

```
PlayerStateStore store = player.GetComponent<PlayerStateStore>();
store.SetSkin("Demo");
store.SetAppearanceColors(headColor, bodyColor, handsColor);
store.SetFacingFromMovement(new Vector2(1f, 0f));
```

El estado incluye:

Dato	Efecto
`Position`	**Posición lógica sincronizada con Rigidbody2D.**
`RoleId`	**Rol actual; si falta, se normaliza a `visitor`.**
`SkinId`	**Skin visual; por defecto `Demo`.**
`FacingDirection`	**Una de cuatro orientaciones: abajo-derecha, abajo-izquierda, arriba-derecha, arriba-izquierda.**
Duración de fotograma	**Tiempo que se muestra cada imagen de la animación.**
Fotogramas por animación	**Cantidad de imágenes que se recorren en cada ciclo.**
`HeadColor`, `BodyColor`, `HandsColor`	**Colores agrupados de la apariencia.**
`ColliderSize`, `ColliderOffset`	**Dimensiones y desplazamiento del BoxCollider2D.**
`YSortEnabled`	**Activa o desactiva el orden visual por posición Y.**
`PlayerId`, `IsMoving`	**Identidad de replicación y estado de movimiento actual.**

Acciones como `SetSkin`, `SetFacingFromMovement`, `SetPosition` y `SetAppearanceColors` pasan por `PlayerStateReducer`. Los cambios solo notifican `StateChanged` cuando el estado resultante difiere del actual.

4. Configurar nombre, rol y progreso de sesión

`GameSessionStore.Instance` mantiene el estado de sesión mientras la aplicación cambia de escenas. Para observar cambios, suscríbete a `StateChanged`; para modificarlos, usa las acciones de sesión o los métodos públicos del store.

El estado de sesión contiene:

- `PlayerName` y `SelectedRoleId`.
- `Level` y `Experience`.
- Métodos como `StartVisitorSession`, `SelectRole`, `AddExperience` y `SetExperience`.

`PlayerStateRuntimeApplier` inicializa el `RoleId` del avatar local desde el rol seleccionado en `GameSessionStore`. `GameplayHUDSessionBinder` lee el store y muestra nombre, nivel, rol y progreso de experiencia. Si no hay nombre de sesión, el HUD muestra `Visitante`.

El store de sesión usa `DontDestroyOnLoad`, pero eso solo conserva los valores entre escenas dentro de la ejecución actual. No equivale a guardado permanente en disco ni a persistencia entre partidas.

5. Configurar el aspecto por capas

Los assets de apariencia están en `Assets/_SocialProyectoCenigraf/Data/Player/Skins/`:

- `PlayerSkinCatalog` contiene las definiciones disponibles.
- Cada `PlayerSkinDefinition` tiene un `Skin Id` y sprites para sombra, piernas, cuerpo, accesorio, manos y cabeza.
- `PlayerLayerOrderProfile` determina qué capas quedan delante según la animación y dirección.

Cada tipo de animación tiene cuatro fotogramas. El orden de los cuatro grupos en cada arreglo es:

1. `IdleFront` (fotogramas 0-3).
2. `WalkFront` (4-7).
3. `IdleBack` (8-11).
4. `WalkBack` (12-15).

Los grupos se definen en `PlayerAnimationType`. `PlayerLayeredSkinController` determina frente o espalda a partir de la orientación, refleja horizontalmente `LayeredSkin` para mirar a izquierda o derecha, aplica el orden y actualiza todos los renderers. La sombra permanece blanca; cabeza, cuerpo/accesorios/piernas y manos toman los colores agrupados del estado.

Para crear la definición de demostración desde el Editor existe **Tools >Cenigraf >Player >Configure Demo Layered Skin**. Esta herramienta prepara la skin `Demo`, el catálogo, el perfil de orden y el jugador de la escena activa a partir de los sprites esperados por el script.

6. Usar controles táctiles

`MobileControls` crea el joystick y el botón de interacción en dispositivos móviles, respeta el área segura de pantalla y busca periódicamente al jugador que acepta entrada local. El botón ejecuta la interacción contextual más cercana y también sirve para avanzar o confirmar ciertos diálogos.

El HUD táctil aparece cuando hay un jugador local activo y se oculta durante viajes. El joystick se desactiva mientras hay diálogos o ventanas modales. No es necesario agregar un joystick separado al prefab del jugador.

7. Distinguir jugador local y réplica remota

`PlayerReplicationContext` define la autoridad del objeto:

- `OfflineLocal`: jugador local sin autoridad de red conectada.
- `LocalOwner`: jugador local propietario.
- `RemoteReplica`: objeto que no debe aceptar input local.

`PlayerMovementController` consulta `AcceptsLocalInput`. `PlayerReplicationBridge` puede capturar un `PlayerReplicatedStateData` o aplicar un snapshot a una réplica remota que tenga un ID coincidente. `Player_Remote.prefab` es una base visual/operativa para esa réplica; esto no implica que exista transporte de red configurado.

Valores iniciales y límites

- Skin inicial: `Demo`.
- Rol por defecto del estado del jugador: `visitor`.
- Duración por defecto de fotograma: 300 ms y cuatro fotogramas por animación; el prefab de escena puede sobrescribir estos valores.

- Colores y collider usan normalización del estado para sustituir valores no configurados por defaults seguros.
- Nombre, rol, nivel y experiencia se conservan entre escenas mientras la ejecución esté activa; no son persistencia de cuenta.
- El movimiento no resuelve pathfinding, gravedad ni navegación. Se ejecuta con Rigidbody2D cinemático y consultas de colisión.
- Inventario, misiones, diálogo y recompensas son subsistemas separados que pueden consultar el rol o interactuar con el jugador.

Solución de problemas

Síntoma	Qué revisar
El jugador no se mueve	**Que el action map tenga `Player/Move`, que `PlayerMovementController` esté activo, no haya un bloqueo global y el objeto acepte input local.**
El movimiento atraviesa o se atasca en paredes	**Revisa Rigidbody2D, collider no trigger, `Collision Mask`, capas de colisión y `Skin Width`.**
El personaje se mueve pero no anima	**Verifica `PlayerStateStore`, `PlayerLayeredSkinController`, `Skin Catalog`, `Skin Id` y los arreglos de 16 sprites por capa.**
La orientación o el orden de capas se ve mal	**Revisa dirección, `Layer Order Profile`, renderers y el transform `LayeredSkin`.**
El HUD muestra Visitante o nivel 1	**Comprueba que se haya iniciado sesión o actualizado `GameSessionStore` antes de revisar el HUD.**
El jugador de una escena no funciona después de teleport	**La escena destino debe contener su propio jugador local activo con `PlayerMovementController` y `PlayerStateStore`.**
El objeto remoto responde al teclado	**Configura su `PlayerReplicationContext` como `RemoteReplica` y confirma que los sistemas consulten `AcceptsLocalInput`.**

Código y assets de referencia

- Prefabs: `Assets/_SocialProyectoCenigraf/Prefabs/Entities/PlayerDemo/Player.prefab` y `Player_Remote.prefab`.
- Input actions: `Assets/InputSystem_Actions.inputactions`.
- Movimiento: `Scripts/Player/Movement/PlayerMovementController.cs`.
- Estado Redux del avatar: `Scripts/Player/State/Player/PlayerStateData.cs`, `PlayerAction.cs`, `PlayerStateReducer.cs` y `PlayerStateStore.cs`.
- Inicialización/aplicación del estado: `PlayerStateRuntimeApplier.cs`.
- Skin y render por capas: `Scripts/Player/Visual/`.
- Sesión, rol, nivel y experiencia: `Scripts/Player/Session/State/`.
- Controles táctiles: `Scripts/Mobile/MobileControls.cs` y `MobileJoystick.cs`.
- HUD de sesión: `Scripts/UI/GameplayHUDSessionBinder.cs` y `GameplayHUDView.cs`.
- Límite de replicación: `Scripts/Player/Replication/`.